# Relationship Mode & Couple Tools Official Companion Contract v1

Shared container, consent law, room architecture, state machines, object map, payload packets, and write-back governance for the relationship-side evolution of the app.

**Purpose.** This contract defines what happens after two users intentionally enter relationship mode. The product no longer behaves like a dating surface. It evolves into a dual-user environment for couple awareness, repair, rituals, progress tracking, and shared growth while preserving private lanes, consent boundaries, and workbook score law.

This document is calibrated to the current Genesis OS system. It assumes the existing readiness, compatibility, drift, Sanctuary, ExsAnonymous, cluster, shadow, genome, and score-engine contracts remain authoritative in their own lanes. Relationship mode consumes and contextualizes those systems; it does not replace them.

## 1. Executive summary

Relationship mode is the platform's intentional second life-cycle. Two users can choose to move from single-user dating mode into a shared couple environment. Once they do, the app should not feel like the same product with a shared chat bolted onto it. It becomes a governed relationship container with its own rooms, permissions, pulse rituals, repair flows, private mirrors, and dual-write data model.

Core requirement: every couple feature must clearly separate four data lanes.

- Shared couple lane - visible to both members and written to the couple container.
- Private self lane - visible only to the originating user and never exposed to the partner without explicit consent.
- Derived system lane - internal operational data used for scoring, patterning, readiness support, forecast support, moderation, and AI routing.
- Protected review lane - restricted support/admin data used only where policy explicitly allows review, moderation, or recovery operations.

> **Design stance.** When a couple enters relationship mode, the app should evolve from a search-and-match posture into a home-and-growth posture. Users are not just continuing to date inside the app; they are entering a shared digital relationship environment.

## 2. Scope and non-goals

| Area | In scope | Out of scope for this contract |
|---|---|---|
| Relationship activation | Couple formation, acceptance, role creation, couple handle, room theme selection, mode transition packets | Public social feed rules and scoreboard economy logic |
| Couple tools | Pulse checks, repair flows, couple space, rituals, reflections, archive, shared goals, private mirrors | Scoreboard social ranking mechanics |
| Data governance | Shared/private object map, consent matrix, retention classes, write-backs, auditability | Ad monetization logic and future marketplace extensions |
| AI use | Couple-support AI permissions inside governed modules | Free-form always-on chat outside module permissions |

## 3. Relationship-mode product model

Relationship mode has three practical layers:

- Container layer - the couple object, handles, room metadata, state, consent records, access policy, and canonical shared archive.
- Experience layer - themed rooms, pulse rituals, repair modules, couple dashboards, shared timelines, and privacy-aware partner surfaces.
- Interpretation layer - governed use of clusters, drift, shadow, grief, anger, and genome features to support guidance without breaking private boundaries.

## 4. Canonical state machine

| State | Description | Entry trigger | Exit trigger |
|---|---|---|---|
| proposed | One user initiates relationship mode invitation | invitation_created | accepted, expired, withdrawn, declined |
| pending_dual_consent | Invitation accepted in principle; required permissions not fully granted | invitation_accepted | consent_complete or consent_fail |
| theme_selection | Couple chooses home theme / room skin / couple handle | dual_consent_complete | theme_published |
| active_couple | Relationship mode fully live | theme_published | paused, breakup_mode, archived |
| cooldown_repair | High-friction or protected repair state | system or user action | return_to_active or breakup_mode |
| paused | Users keep link but disable some active surfaces | pause_requested | resume_active or breakup_mode |
| breakup_mode | Relationship link ended; post-link flows active | mutual or asymmetric end event | closure_complete |
| archived | Historic couple record retained under policy | closure_complete | restored only by policy-approved path |

## 5. Shared/private object map

The couple architecture should be implemented as explicit data objects rather than inferred from chat history. Below is the minimum object map.

| Object | Lane | Owner | Visible to partner | Writes to system |
|---|---|---|---|---|
| couple_container | shared | system/couple | yes | yes |

| | | | | |
|---|---|---|---|---|
| couple_profile | shared | both via consented edits | yes | yes |
| couple_handle | shared | couple | yes | limited |
| room_theme_profile | shared | both | yes | no |
| pulse_check_session | shared | couple | yes by rules | yes |
| repair_session | shared | couple | yes | yes |
| private_reflection_entry | private self | single user | no | limited derived only |
| private_partner_note | private self | single user | no | no unless converted by consent |
| couple_goal_object | shared | couple | yes | yes |
| intimacy_or_boundary_pref | private self + derived | single user | no raw; only approved deltas | yes |
| couple_archive_item | shared or private-tagged | depends on source | depends on scope | yes |
| consent_record | protected/system | system | only summary state | yes |
| relationship_transition_log | protected/system | system | no | yes |

### Object law:

- No private self object may leak into the shared lane unless a specific conversion action is taken and logged.
- Derived-system write-backs may summarize risk, support need, or pattern posture without exposing raw private text.
- Couple tools should consume shared objects first and private objects only through module-specific permissions.
- When uncertainty exists, treat the object as private and require a deliberate share action.

## 6. Consent matrix

Relationship mode requires more than a simple accept button. It requires a layered consent contract.

| Consent type | Required at activation | Revocable | Effect of revocation |
|---|---|---|---|
| relationship_link_consent | yes | yes | couple state can pause or enter breakup mode |

| | | | |
|---|---|---|---|
| shared_space_creation | yes | no after creation; can archive later | space remains but may be hidden |
| shared_pulse_visibility | yes | yes | future pulse summaries limited or private-first |
| repair_module_participation | optional until used | yes | user can decline or exit repair flows |
| AI_couple_support | optional | yes | AI guidance disabled or reduced |
| private_to_shared_conversion | never blanket; per action | n/a | conversion does not occur |
| partner_pattern_surface | optional and bounded | yes | hide partner-safe insights |
| memory_archive_coauthoring | optional | yes | future entries scoped individually |

**Consent rule.** There is no global blanket consent for one partner to view the other's private reflections, private ExsAnonymous activity, private Sanctuary logs, or protected support artifacts. Any system design implying this should be rejected.

## 7. Couple handle and couple-space identity

On activation, the pair receives a governed shared identity object.

- couple_handle: human-readable couple identifier used inside the shared space; distinct from individual usernames and RS codes.
- couple_theme_profile: selected aesthetic shell for the shared home, rooms, accent palette, iconography, and ceremonial transitions.
- couple_motto / relationship intention line: optional short text visible in the shared space.
- home_state: which rooms are unlocked, active, muted, or in cooldown.

The handle and theme are not cosmetic only. They are metadata keys for UI routing, room skinning, ritual entry animations, storage segmentation, and notification language.

## 8. Room architecture

The couple side should feel like a space the relationship grows inside. Because exact final creative labels may evolve, engineering should implement room families with stable functional IDs and separately skinnable display names/themes.

| Functional room id | Purpose | Default visibility | Examples of data |
|---|---|---|---|

| | | | inside |
|---|---|---|---|
| home_room | overview / dashboard | shared | relationship status, recent pulse, next ritual, growth streak |
| pulse_room | routine emotional check-ins | shared by module rules | pulse sessions, sentiment summaries, unresolved flags |
| repair_room | structured rupture and repair work | shared | repair sessions, acknowledgement steps, action commitments |
| memory_room | shared archive and milestones | shared + scoped items | anniversaries, wins, letters, selected reflections |
| growth_room | goals, habits, rituals, improvement paths | shared | couple goals, streaks, commitments, learning modules |
| sanctuary_bridge | private-to-couple support bridge | mixed | partner-safe support prompts, guided transitions |
| intimacy_room | care / closeness rituals and relationship nurture | shared by permission | connection prompts, date rituals, closeness plans |
| cooldown_room | temporary lower-intensity conflict-safe space | restricted shared | timeouts, re-entry timers, softened prompts |

### Engineering room-theme requirement:

- Display theme names, background art, and decor variants must be skinnable configuration values, not hard-coded logic paths.
- Every room theme selection should write both the design token set and the functional room mapping.
- Users may change themes without rewriting historical room data.
- The app's visual evolution from dating mode to couple mode should be represented as a mode transition packet, not just a front-end flag.

## 9. Relationship-mode transition packet

When both users elect relationship mode, the system should emit a canonical transition packet.

| Field | Type | Notes |
|---|---|---|
| event_name | string | relationship_mode_activated |
| source_user_ids | array[2] | ordered but role-neutral |

| | | |
|---|---|---|
| couple_id | uuid | new canonical couple key |
| compatibility_snapshot_id | uuid | score snapshot at activation time |
| readiness_snapshot_ids | array[2] | individual snapshots at activation |
| consent_bundle_id | uuid | links all activation consent records |
| couple_handle | string | final accepted handle |
| theme_profile_id | string/uuid | selected couple-space theme |
| mode_version | string | to support migration |
| created_rooms | array | functional room ids provisioned |
| initial_visibility_profile | json | what shared surfaces start enabled |

# 10. Couple tool modules

## 10.1 Pulse checks

Pulse checks are the routine heartbeat of relationship mode. They are not generic mood emojis. They are structured dual-input sessions used to surface connection, pressure, friction, repair need, and support timing.

| Pulse object | Required | Shared or private | Write-back posture |
|---|---|---|---|
| pulse_session_id | yes | shared | system + shared |
| prompt_set_id | yes | shared | shared |
| response_a / response_b | yes | private until both complete; then module-specific shared surface | derived + shared summary |
| alignment_delta | yes | derived | system |
| repair_flag | conditional | derived | system + optional shared alert |
| support_prompt_id | conditional | shared | shared |
| cooldown_recommendation | conditional | derived | system |

Pulse capture rules:

- If only one user responds, the system stores a pending pulse partial and does not reveal comparative output.
- Raw pulse text remains private unless the pulse design explicitly uses shared free-text with consent.
- The shared surface should generally show a regulated summary, not one partner's unfiltered private wording.
- Repair flags can write to the repair queue without exposing blame language.

### 10.2 Repair modules

Repair modules are structured workflows for rupture acknowledgement, emotional translation, accountability, and action closure. They should never be implemented as a loose free-for-all chat wrapper.

| Repair phase | State | Output |
|---|---|---|
| rupture_detected | queued | repair session candidate |
| entry_consent | active | both users accept module entry |
| facts_and_feelings | active | parallel guided inputs |
| translation_and_acknowledgement | active | partner-safe summaries and acknowledgement objects |
| commitment_and_next_step | active | repair commitments with due windows |
| closure_review | complete or reopen | closed session or reopened issue |

### 10.3 Private mirrors inside couple mode

Relationship mode still needs private self-reflection spaces. The product becomes a couple environment, not surveillance software. Each partner keeps private mirrors for self-check, clarification, drafts, and emotional sorting before converting anything into shared dialogue.

- Private mirror entries can influence derived support state, but not expose raw language.
- A private entry can be manually converted into a shared note, repair starter, ritual prompt, or archive artifact.
- Private mirror activity may seed a support suggestion to the author only.

## 11. Relationship data model

| Data family | Key fields | Why it exists |
|---|---|---|
| couple identity | couple_id, handle, theme_profile, status, created_at | creates durable shared container |

| | | |
|---|---|---|
| membership / roles | user_id, couple_id, role_status, joined_at, ended_at | tracks participation without ownership asymmetry |
| consent records | consent_type, scope, granted_at, revoked_at, source | governs lawful feature behavior |
| room registry | room_id, room_family, visibility, state, skin_token | supports room-specific routing |
| pulse data | session_id, prompt_set, result_state, support_flags | maintains couple heartbeat |
| repair data | repair_id, issue_type, stage, commitments, closure_state | structures rupture-to-repair flow |
| archive and memory | item_id, scope, source_module, visibility, tags | stores relationship history cleanly |
| private self notes | entry_id, author, module_source, convertable_flag | preserves private lane |
| relationship health support | trend flags, drift interlocks, support posture | feeds guidance without overwriting score law |

## 12. Shared/private separation rules

- A partner may never read the other's private mirror, private Sanctuary session, private ExsAnonymous room, or private draft unless an explicit share conversion occurs.
- Shared summaries must be generated from allowed fields only. The summarizer cannot ingest prohibited private text for partner display.
- Derived system features may use private source data to decide support posture, but partner surfaces must receive only policy-approved abstractions.
- Any partner-visible insight that originated from private data must pass through partner-safe display logic and consent checks.

## 13. ExsAnonymous and Sanctuary inside relationship mode

ExsAnonymous remains part of Sanctuary. In couple mode, ExsAnonymous does not become partner-readable confession storage. It remains private by default, with only governed downstream effects allowed.

| Source module | Raw data partner-visible? | Can influence couple support? | Allowed couple-facing output |
|---|---|---|---|
| Private ExsAnonymous session | no | yes, bounded | partner-safe support suggestion only if policy allows |

| | | | |
|---|---|---|---|
| Private Sanctuary reflection | no | yes, bounded | self-directed prompt or neutral repair suggestion |
| Shared couple ritual in Sanctuary bridge | yes by design | yes | shared output object |
| Last Light / grief work | no raw | yes, strongly bounded | care posture suggestion, not content leak |

## 14. Drift, clusters, shadow, anger, and grief interlocks

Relationship mode should not invent a new diagnostic empire. It should consume governed companion systems through clear permissions.

| System | May influence couple tools | May expose raw internals to partner | Typical use |
|---|---|---|---|
| drift | yes | no | suggest pulse, cooldown, or repair timing |
| clusters | yes | no raw | identify recurring repair families or support routes |
| shadow activation | yes, carefully | no raw labels | increase support guardrails and AI restraint |
| anger hex | yes | partner-safe only | cooldown, repair pacing, safety-aware prompts |
| E_GRIEF | yes | partner-safe only | care posture and pacing |

## 15. Consent-aware AI permissions

| AI action | Allowed? | Conditions |
|---|---|---|
| summarize shared pulse session | yes | both users completed pulse and module permits summary |
| read one partner's private mirror for partner-facing output | no | never |
| generate repair prompt | yes | active repair module with consent |
| infer support pacing from grief/anger/drift signals | yes | must remain partner-safe and policy-bounded |
| quote private ExsAnonymous text | no | never |

| | | |
|---|---|---|
| to partner | | |
| create shared archive recap | yes | from shared lane content only |
| coach one user privately about couple issue | yes | private lane only; no partner leak |

## 16. Room-specific payload envelopes

Every major couple module should emit structured packets. Below is the minimum pattern.

| Packet | Core fields | Why devs need it |
|---|---|---|
| couple_room_opened | event_id, couple_id, room_id, actor_id, timestamp, ui_state | analytics and state restoration |
| pulse_submitted | session_id, author_id, response_scope, completion_state | dual-input synchronization |
| repair_started | repair_id, trigger_source, consent_state, issue_family | structured repair routing |
| private_to_shared_conversion | source_entry_id, author_id, target_object, consent_token | privacy enforcement |
| theme_changed | couple_id, old_theme, new_theme, actor_id | visual/state consistency |
| couple_mode_paused | couple_id, reason_code, visibility_profile | safe deactivation without data loss |

## 17. Breakup / post-link mode

The couple container must support a governed end-state. When the relationship ends, the platform should shift into protected breakup handling instead of deleting reality.

- Freeze shared editing except for approved closure flows.
- Preserve audit-safe archive objects according to retention class.
- Sever shared-room live interactivity unless policy requires a cooling period.
- Route each user back to single-user mode with historical awareness but clean boundary separation.

> **Important.** Breakup mode is not only a social status change. It is a data-boundary event, a permissions event, and often a Sanctuary / ExsAnonymous / grief event.

## 18. Minimum API surface

| Endpoint / action | Purpose |
|---|---|
| POST /relationship/invite | propose relationship mode |
| POST /relationship/consent | submit activation consent bundle |
| POST /relationship/theme | select or update theme profile |
| GET /relationship/home | load shared home container |
| POST /relationship/pulse | create or complete pulse session |
| POST /relationship/repair/start | start governed repair flow |
| POST /relationship/private-share | convert allowed private item into shared object |
| POST /relationship/pause | pause couple mode |
| POST /relationship/end | enter breakup flow |

## 19. QA cases developers must lock

- One partner completes a pulse; the other never responds. Verify no comparative leak appears.
- A private ExsAnonymous entry seeds support posture. Verify raw content never appears to the partner.
- Theme changes do not corrupt room histories or archive IDs.
- Consent revocation hides future partner-safe pattern surfaces without deleting historical protected records.
- Repair module reopened after failed closure maintains versioned history.
- Pause/resume of relationship mode does not orphan the couple container.

## 20. Implementation order

1. Build couple_container, consent records, room registry, and transition packet first.
2. Build shared/private object map enforcement and API guards second.
3. Build home room and pulse room next; they are the minimum viable couple experience.
4. Build repair room after pulse, because pulse is often the entry signal.
5. Build archive, growth, and intimacy rooms after the state/consent core is stable.
6. Integrate drift, cluster, anger, grief, shadow, and Sanctuary bridge outputs only through approved adapters.
7. Add scoreboard/social feed later under a separate contract, not by piggybacking on relationship private data.

## 21. Final build doctrine

Relationship mode should be implemented as a governed couple operating environment. The user experience may feel warm, intimate, and beautifully themed, but the underlying engineering must be strict about consent, lane separation, packet discipline, and write-back law. The platform is allowed to help couples grow; it is not allowed to become voyeur software, accidental therapy cosplay, or a privacy breach with candles.